Research Diary

Caio Simon

July 1, 2026

Current Project Status

July 1, 2026

Ok, it has been a little while since I last wrote about the project, but some progress has been made in the meantime. First, some small administrative updates:

- I found an existing simulator of basketball GM activities called ZenGM. It is very good, very deep, and written in TypeScript. I could adapt it and have that work as my simulator, which would mean I have fewer decisions to make and defend come thesis time, but I find thinking about the problem quite fun. I texted Jeremy, the creator of ZenGM and he said that he did not aim for realism. I don't know if anything I could write would approach ZenGM in quality, as it is a mature project already. It would give me more control over how the world works though.
- Anders is now my supervisor

Ok, now onto the project status.

Player Signing Free agency functionality, in its most basic possible form, has been added. Teams now sign players by offering non-zero salary to them. Players do not have any choice; the first team to bid on them gets them, which will be patched in later rounds.

Initial Player Distribution With the new player functionaly in place, there is no reason to just randomly give players to teams. Instead, there is a salary cap (set at 100, for simplicity) and teams sequentially sign players in the beginning by offering a salary to players with an allocated number of years. Again, the big weakness here is that players do not choose where to go — the first team that bids, gets them.

Player Evolution Players now go through a pre-determined evolution curve, shown here:

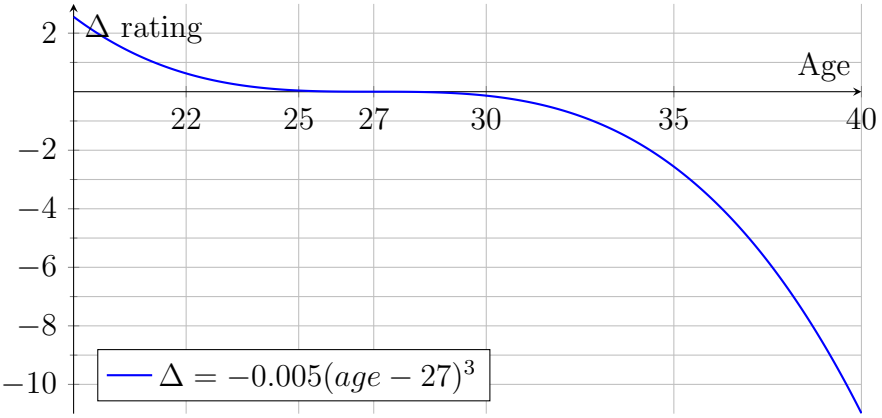

Figure 1: Expected change in player rating as a function of age.

The function I use is:

$$\delta = -0.005 * (\text{age} - 27) * *3 + \epsilon \tag{1}$$

Where ϵ is some noise. For now I hardcode it as $\epsilon = 0.5$, but I have also considered making it a function of the rating (so really good players decline more, or improve faster, but I am not sure about that yet.)

RL training I am also happy to announce that I was able to make an RLlib model train on this environment for the very first time! It was a bit of a pain to get it to work because of dependencies and missing packages (turns out RLlib isn't all that great in Windows) but it trained in the end, which I am happy about.

For the rewards, I used this exponential decay function, with $k = 0.3$:

$$r = e^{-k(\text{position}-1)} \tag{2}$$

The function applies to all teams in playoff positions (1-16, for now). Teams that did not make it also get the same reward function, but first they go through the draft lottery (with the current ruleset), and are assigned rewards based on that. So basically, they can have just as high rewards, but it is uncertain in which position they will land.

I will now work on player retirement and adding new players through some sort of draft or bidding process.

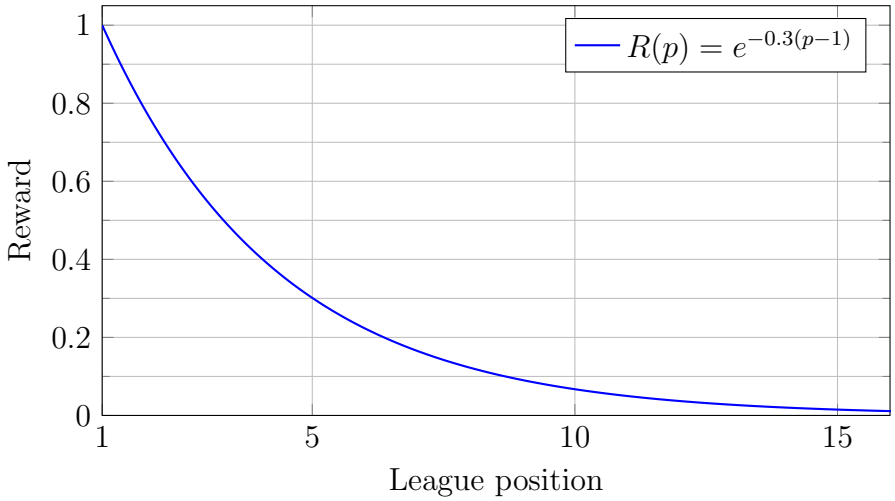

Figure 2: Reward curve for teams in positions 1-16

To-Do (short-term)

- Allow teams to sign players — free agency
  - Initially agents will be able to observe player ratings, to make the simulation easier.
  - Must add preferences for players — past success of a team and how much money they are offering.
- Create trades for players
  - Apparently this is hard to do; the combinatorial space is huge, so I will have to read more about how to do this.
- ✓ Player evolution (easier)
- Player retirement
- New players in the league

To-Do (long-term)

- `PettingZoo` environment where agents can sign players in the off-season.

- `PettingZoo` environment where agents can trade players
- `PettingZoo` environment where agents draft young players.
- Calibration of parameters (veeery long-term)
- Survival analysis for ageing players.

Weakenesses

- Initial talent distribution
- Parameters are guesstimated
- Game simulation is too basic
- Better players should account for a larger portion than end-of-bench players.

Chronological Order

June 20, 2026

Currently, I have managed to build a small simulation of basketball season games. It is an exceedingly simple simulation, but one that provides a strong basis for future work and small RL experiments. Important features are the following:

Player Attributes Players are defined by a single number, which for now I call their **impact** score. I drew from a lognormal distribution with parameters $\mu = 0$ and $\sigma = 1$. I played around with a pareto/power-law distribution as well, but that seemed too extreme for my taste. There were many not-so-great players and some which had absolutely run-away scores. The lognormal has the heavy-tailed aspect but it isn't as extreme as a Pareto distribution. Usually the best player in any given draw is 10 to 15 times better than the average player, which already seems like plenty to me. A pareto distribution usually creates even bigger talent discrepancies.

Talent Distribution Currently, players are simply generated by drawing from my prized lognormal distribution and they are randomly allocated to teams. Obviously, this is **unrealistic**, as talented players may have a preference for playing together (playing for winners) or for teams with capspace — something I will have to figure out how to implement. For now teams cannot trade or sign players. I expect that already when this functionality is implemented that talent will eventually be distributed in a more realistic way.

Game Simulation Currently games are simulated in a very simple manner. A team has 10 players assigned to it, each with their skill score, which I will refer to as s_p as the score of player p. The sum of the scores of all players in the team i will generate S_i, the total score for team i:

$$S_i = \sum_{p \in i} s_p \tag{3}$$

$(p \in i)$ indicates that player p plays for team i.

Games are decided by:

$$P(\text{Team } i \text{ wins}) = S_i - S_j + \epsilon \tag{4}$$

Where $\epsilon \sim \text{Logistic}(0, \sigma_{\text{noise}})$. If $S_i - S_j + \epsilon > 0$, team i is assigned as the winner of the game.

There are many problems with this approach. First, the best player and the end-of-the-bench player all have equal contribution under this scheme, whereas in the NBA it is unlikely the 10th best player would see much gametime. I need to devise a scheme where players get more gametime according to their skill or — which would be even better — I allow my RL model to devise a rotation under some contraints (no player can play more than 48 minutes; 240 minutes have to be apportioned in total). I might have to implement an injury or fatigue related mechanism for that if I want to prevent agents from abusing their best player, or I keep their minutes artificially below a certain threshold (40 minutes, for example).

Second, players cannot be described by a single skill score. In reality we have players that can shoot, players that can pass, those that defend the perimeter, some that defend the paint, slashers, post-up players, mid-range specialists and so on. While it would be lovely to be able to simulate games with that level of granularity, the focus of the project is more on the behaviour of basketball teams, especially as it pertains to tanking. For now, I will forego these complicated

simulations (also in the interest of time) and go ahead with my singular impact score. As the project develops I will also consider breaking this down into offensive ability score and defensive ability score, s_{off} and s_{def}.

Building on from the last point, for now I simply draw players from this lognormal distribution, whose parameters I decided on. In a dream scenario, I would be able to use a latent-factor modelling approach to calibrate this distribution for players. That would make this much more interesting and realistic. Otherwise I could also try to incorporate some ready-made impact metrics like PIPM, xRAPM, LEBRON, and DARKO. Or a statistical model of basketball games, but that seems to take me away from the point of the project already.

Third, and for now last, is the logistic noise that I use, with variance σ_{noise}. This is very important in calibrating the share of matches that are decided by pure skill versus the natural variance in luck.

I simulated 16 seasons while respecting the NBA's scheduling style (more games within a division and conference), and I guesstimated some of the parameters. The win percentage I get with my simulation method is actually decently good for being so simple. Look at the plot below, where I compare the win pct distributions I get in simulation with the actual distribution of win percentages from the 2010-11 season.

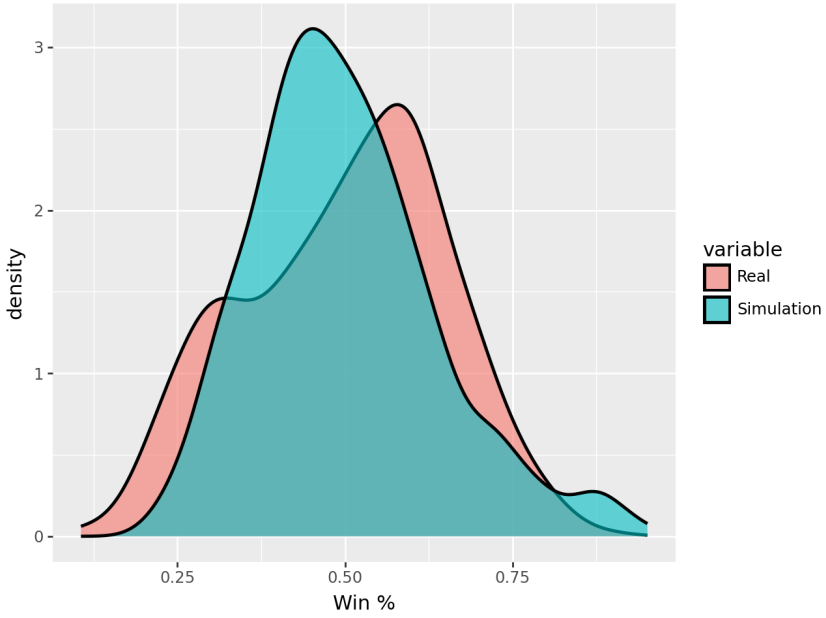

Figure 3: First look at simulated vs real win percentage. I don't get the slightly bimodal distribution yet, but I hope this will happen once teams start to make choices.